

Enhancing LLM Code Generation with Ensembles: A Similarity-Based Selection Approach

Tarek Mahmud

Texas A&M University - Kingsville
Kingsville, Texas, USA
tarek.mahmud@tamuk.edu

Corina Păsăreanu

Carnegie Mellon University
Mountain View, California, USA
pcorina@andrew.cmu.edu

Bin Duan

The University of Queensland
Brisbane, Australia
b.duan@uq.edu.au

Guowei Yang*

The University of Queensland
Brisbane, Australia
guowei.yang@uq.edu.au

Abstract

Ensemble learning has been widely used in machine learning to improve model robustness, accuracy, and generalization, but has not yet been applied to code generation tasks with large language models (LLMs). In this paper, we propose an ensemble approach for LLMs in code generation. Instead of relying on the output of a single model, we generate multiple candidate programs from different LLMs and apply a structured voting mechanism to select the most reliable solution.

For voting, we compute syntactic and semantic similarity using CodeBLEU and behavioral equivalence using CrossHair's differential behavior analysis. By aggregating these similarity scores, we select the program that best aligns with the consensus among the candidates. Our experiments show that our ensemble approach consistently outperforms standalone LLMs on the well-known HumanEval and the more challenging LiveCodeBench datasets, achieving an accuracy of 90.2% and 50.2%, respectively, on the two datasets. In comparison, the best-performing LLM (GPT-4o) has an accuracy of 83.5% and 43.4%, respectively. Furthermore, even when restricted to free open source models, our method achieves an accuracy of 80.5% and 41.6%, respectively, demonstrating the viability of our approach in resource-constrained settings.

Keywords

Large Language Model, Code Generation, Ensemble Learning

ACM Reference Format:

Tarek Mahmud, Bin Duan, Corina Păsăreanu, and Guowei Yang. 2026. Enhancing LLM Code Generation with Ensembles: A Similarity-Based Selection Approach. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26)*, April 12–18, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3744916.3773143>

*Corresponding author



This work is licensed under a Creative Commons Attribution 4.0 International License. *ICSE '26, Rio de Janeiro, Brazil*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2025-3/26/04

<https://doi.org/10.1145/3744916.3773143>

1 Introduction

Large Language Models (LLMs) [12] have significantly advanced automated code generation, enabling models to generate functionally correct program from natural language prompts. Recent models, such as GPT-4 [8], CodeLlama [4], and DeepSeekCoder [5], have demonstrated strong performance on benchmark datasets, making them increasingly popular in software development workflows. These models leverage large-scale training on diverse code repositories, allowing them to generate code for a wide range of programming problems across different programming languages. Their ability to generate, complete, and refactor code has led to widespread adoption in software engineering, competitive programming, and AI-assisted development environments.

However, despite these advancements, LLMs are not infallible and may produce incorrect or suboptimal code, leading to syntactic errors, logic mistakes, or missing edge case handling [16]. The reliability of LLM-generated code depends heavily on the complexity of the problem, the quality of training data, and the model's ability to generalize beyond its training distribution. Even state-of-the-art models such as GPT-4 achieve only around 82% accuracy on functional correctness on standard coding benchmarks such as HumanEval, while open-source models such as CodeLlama achieve much lower accuracy (42-50%) [13, 53]. These discrepancies highlight the performance gap between proprietary and open-source models, making it challenging to ensure consistent performance across different LLMs.

A key challenge in LLM-based code generation is the unpredictability of failure modes. Studies have found that LLMs tend to favor syntactically plausible solutions, even when they are functionally incorrect [36]. Moreover, LLMs struggle with long-range dependencies in code, leading to issues when reasoning about complex data structures or multi-step algorithms. OpenAI Codex and GitHub Copilot evaluations have also revealed that LLMs can introduce security vulnerabilities, such as using deprecated APIs, generating unsafe cryptographic implementations, or failing to handle edge cases properly [42]. These limitations make post-generation validation essential, reinforcing the need for approaches that can systematically assess and filter correct solutions before deployment in real-world applications.

Ensemble learning has been widely used in machine learning to improve model robustness, accuracy, and generalization [39]. Traditional ensembling techniques, such as bagging, boosting, and

stacking, have shown success in various domains, including image classification [24, 33], natural language processing [14, 45], and anomaly detection [10, 17]. However, they have not been extensively explored in LLM-based code generation. Prior research suggests that generating multiple solutions, albeit with a single model, and selecting the best candidate can significantly improve performance, as seen in OpenAI’s Codex study, where pass@100 accuracy was far higher than pass@1 [16]. Inspired by these previous works, we propose EnsLLM, an ensemble-based approach that combines the outputs of multiple LLMs and selects the most reliable solution using a structured voting mechanism. Instead of relying on a single model’s output, EnsLLM uses multiple candidate programs and applies a novel similarity-based ranking that aims to evaluate syntactic, semantic, and behavioral correctness.

Specifically, EnsLLM integrates CodeBLEU [44], a popular metric for the evaluation of program produced by LLMs, and differential analysis to assess the reliability of the generated program. CodeBLEU is used to measure syntactic and semantic similarity between pairs of candidate programs, such that syntactically and semantically similar and logically aligned solutions receive higher rankings. As CodeBLEU uses only static information about a program, we also investigate a complementary *execution-based* differential analysis to detect behavioral inconsistencies between candidate pairs and generate counterexamples where the two candidates produce different outputs. In our work, we define a behavioral similarity metric based on the property-based testing tool CrossHair [3], which is used for differential analysis (but other differential analyses can also be used [34, 40]). Combining these two metrics, EnsLLM computes pair-wise scores for all the candidates, aggregates these scores for each candidate, and selects the candidate with the highest score as the output program of the ensemble.

We evaluate EnsLLM on two well-established code generation benchmarks: HumanEval [16] and LiveCodeBench [51]. Our results show that EnsLLM consistently outperforms individual LLMs, achieving 90.2% accuracy on HumanEval and 50.2% on LiveCodeBench, surpassing the best standalone model in both datasets. Additionally, even when restricted to using only free and open-source LLMs, EnsLLM achieves 80.5% on HumanEval and 41.6% on LiveCodeBench, proving its viability in resource-constrained environments.

The key contributions of this paper are as follows:

- We propose an ensemble-based approach for LLM-based code generation, leveraging syntactic, semantic, and behavioral similarity for improved reliability.
- We propose a novel voting mechanism that integrates CodeBLEU and CrossHair to assess syntactic similarity, semantic alignment, and functional correctness of generated program.
- We conduct extensive experiments on HumanEval and LiveCodeBench, demonstrating that EnsLLM significantly outperforms standalone LLMs. We analyze failure cases and demonstrate that correct programs reinforce each other, supporting our assumption that independent LLMs are less prone to making identical mistakes.
- We show that EnsLLM remains effective even when restricted to free and open-source LLMs, making it a viable solution for environments with limited access to proprietary models.

2 Background

2.1 Ensemble Learning

Ensemble learning enhances predictive accuracy by combining multiple models to reduce variance, bias, and overfitting. Key techniques include bagging, boosting, stacking, voting, etc.

Bagging [11] (Bootstrap Aggregating) trains multiple models on different dataset subsets sampled with replacement and combines their predictions through averaging (regression) or majority voting (classification). Random Forest, an ensemble of decision trees, exemplifies this approach, reducing variance and improving stability. Boosting [25] trains models sequentially, with each correcting the errors of its predecessor. Algorithms like AdaBoost, GBM, and XGBoost iteratively refine predictions, improving accuracy by focusing on hard-to-classify cases. Stacking [47] combines diverse base models using a meta-learner, which determines the best way to integrate predictions for enhanced generalization. Weighted averaging assigns different contributions to models based on performance, further improving stability and accuracy.

Voting [32] ensembles aggregate predictions from multiple models through majority voting (hard voting) or by averaging predicted probabilities (soft voting). This method is particularly effective when individual models have complementary errors, resulting in a more robust final decision.

Ensemble methods mitigate individual model limitations, making them effective for classification, regression, and generative tasks. This paper proposes EnsLLM a voting-based ensemble of LLMs on the code generation task.

2.2 CodeBLEU

CodeBLEU [44] is a specialized metric designed to evaluate the quality of automatically generated code, building upon the traditional BLEU (Bilingual Evaluation Understudy) metric by addressing its shortcomings in the context of programming languages. Standard BLEU, widely used in natural language processing, measures n-gram precision—essentially the overlap of word sequences between a generated text and a reference text—providing a simple yet effective way to assess lexical similarity. However, this approach lacks the depth required to understand the structured nature of code, where syntactic correctness and semantic intent are paramount beyond mere token matching. CodeBLEU overcomes these limitations by integrating a multifaceted evaluation framework that captures both the syntactical, semantical and logical aspects of programming, making it particularly suited for assessing code generation tasks.

CodeBLEU’s evaluation includes four elements. First, n-gram precision captures lexical similarity by comparing token sequences (e.g., keywords, identifiers) between generated and reference program. Second, weighted n-gram matching adjusts this by assigning higher importance to key tokens (e.g., function names over punctuation), refining lexical accuracy. Third, Abstract Syntax Tree (AST) similarity evaluates syntactic structure, comparing hierarchical representations of program constructs like loops or function calls to ensure syntactical consistency. Fourth, data flow similarity analyzes variable transformations and dependencies, verifying logical equivalence even across differing implementations. The final CodeBLEU score is a weighted sum of these components, balancing lexical, syntactic, and semantic fidelity.

This multi-faceted metric enables precise ranking and filtering of program outputs, distinguishing high-quality implementations from flawed ones despite superficial similarities. As background, CodeBLEU’s comprehensive approach lays the groundwork for our method, which builds on its strengths to enhance ensemble-based selection of accurate and functionally reliable solutions.

2.3 CrossHair

CrossHair [3] is a Python tool that automatically finds counterexamples to assertions in program by using symbolic execution. It works by analyzing the logic of functions, preconditions, and invariants, exploring different execution paths to detect errors that conventional testing might miss. CrossHair integrates Python’s type hints and assert statements to validate function behavior dynamically, making it a useful tool for debugging and verifying correctness.

The `diffbehavior` feature in CrossHair allows users to compare different versions of a function by executing them with the same inputs and identifying cases where their outputs diverge. It systematically explores possible inputs and reports counterexamples—inputs that lead to unexpected or incorrect behavior in one version but not the other. By highlighting these discrepancies, `diffbehavior` helps developers detect unintended changes, regressions, or subtle logic errors. This feature is particularly useful when refactoring or optimizing complex functions, ensuring that modifications do not introduce new bugs while maintaining expected behavior.

3 Approach

We propose an ensemble approach for LLM-based code generation that generates multiple candidate programs from different LLMs and applies a voting mechanism to select the most reliable solution. The main challenge we had to address was: *How to do the voting?* We note that although LLMs do not normally compute confidence values, these values can be inferred from the probabilities assigned to each token, during generation. However, as different LLMs are not calibrated, we can not simply use their confidence values for voting. To address this challenge, we propose a voting mechanism that is based on *syntactically, semantically, and behaviorally meaningful* pairwise comparison between the programs.

Figure 1 provides an overview of our approach. First, it filters out syntactically invalid programs and then selects the most representative solution using similarity-based voting. Since LLM-generated programs often contain hallucinations, incomplete structures, or misplaced tokens, syntactic filtering is a necessary step to prevent misleading similarity calculations and focus the selection process on functionally viable candidates.

To evaluate the similarity between candidate programs, we employ two complementary similarity measures: CodeBLEU for syntactic and semantic similarity and behavioral similarity measured with differential analysis. CodeBLEU captures lexical, structural, and data-flow relationships between programs, while differential analysis (CrossHair) detects functional inconsistencies by generating counterexamples where two programs produce different outputs. By aggregating these similarity scores, our structured voting mechanism selects the most representative program, as the output of the ensemble.

3.1 Syntactic and Semantic Similarity

For syntactic and semantic similarity, we use CodeBLEU. As mentioned, CodeBLEU extends traditional BLEU by incorporating program-specific features to evaluate the quality and similarity of generated programs beyond simple token overlap. Unlike BLEU, which primarily focuses on n-gram matching in natural language, CodeBLEU introduces additional dimensions tailored for source code: lexical n-gram matching (`n_gram_weight`), syntactic structure matching (`syntax_weight`), semantic data-flow analysis (`dataflow_weight`), and token weighting (`token_weight`). In our work, we only use `syntax_weight` and `dataflow_weight`, i.e., assign non-zero weight, as we are primarily concerned with the syntactical and semantic similarity of programs rather than their textual resemblance. By focusing on these two measures, we aim to reduce the influence of superficial lexical similarities while still capturing key syntactical and semantic patterns.

- **Syntactic Structure Matching (`syntax_weight`):** Uses Abstract Syntax Tree (AST) matching to compare the syntactical organization of programs. Programs with similar syntax trees are syntactically equivalent, even if their tokens differ. This prevents the penalization of programs with different formatting or minor syntactical variations.
- **Semantic Data-Flow Matching (`dataflow_weight`):** Analyzes data-flow dependencies between variables to assess functional similarity. Two programs are considered equivalent if they manipulate variables and data in the same way, regardless of syntax. This ensures that semantically identical but lexically different implementations receive high similarity scores.

```

1 def binary_search_iterative(arr, target):
2     left, right = 0, len(arr) - 1
3     while left <= right:
4         mid = (left + right) // 2
5         if arr[mid] == target:
6             return mid
7         elif arr[mid] < target:
8             left = mid + 1
9         else:
10            right = mid - 1
11    return -1

```

Listing 1: Iterative Binary Search Implementation

```

1 def binary_search_recursive(arr, target, left, right):
2     if left > right:
3         return -1
4     mid = (left + right) // 2
5     if arr[mid] == target:
6         return mid
7     elif arr[mid] < target:
8         return binary_search_recursive(arr,
9                                         target, mid + 1, right)
10    else:
11        return binary_search_recursive(arr,
12                                        target, left, mid - 1)
13
14 def binary_search(arr, target):
15    return binary_search_recursive(arr,
16                                    target, 0, len(arr) - 1)

```

Listing 2: Recursive Binary Search Implementation

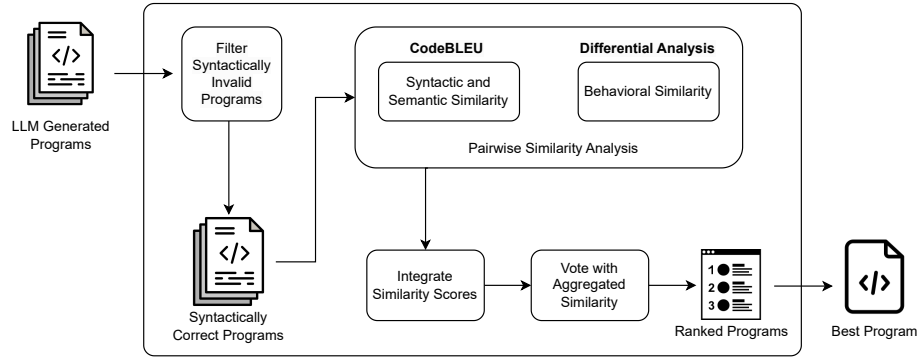


Figure 1: Overview of EnsLLM.

CodeBLEU plays a crucial role in our ensemble approach by performing pairwise comparisons between generated programs, ultimately ranking them based on aggregated similarity scores. Given a binary search task, correct implementations—such as the iterative (Listing 1) and recursive (Listing 2) approaches—differ structurally but yield the same results. In contrast, incorrect implementations, such as the linear search (Listing 3), introduce inefficiencies or errors that deviate from the intended algorithmic behavior (sometimes small LLMs make this type of mistake). When performing pairwise similarity checks, CodeBLEU’s syntax-based metric (derived from Abstract Syntax Trees) provides the iterative and recursive binary search implementations a moderate similarity score of 0.6, while its data-flow component captures their logical equivalence with a score of 1.0. These correct implementations reinforce each other in pairwise comparisons, ensuring that they receive consistently high rankings despite their syntactic differences. In contrast, the incorrect implementations receive lower scores when compared to either correct version—for instance, the linear search receives only a syntax similarity of 0.2 with Listing 1 and 0.3 with Listing 2 and a data flow similarity of 0.3 with Listing 1 and 0.2 with Listing 2. This low score of the incorrect implementation ensures that correct implementations complement each other syntactically and semantically.

```

1 def linear_search(arr, target):
2     for i in range(len(arr)):
3         if arr[i] == target:
4             return i
5     return -1

```

Listing 3: Linear Search Implementation (Incorrect, as the task requires a Binary Search algorithm)

A key assumption in our approach is that LLMs do not make identical mistakes when generating an incorrect program. This naturally reduces the similarity between correct and incorrect solutions, such as between the binary search implementations (Listings 1 and 2) and the linear search (Listing 3), as well as between two incorrect implementations, preventing erroneous programs from being ranked highly. Moreover, similar implementations will complement each other by reinforcing their shared syntactic and semantic strengths, further boosting their collective score within the ensemble.

3.2 Behavioral Similarity

Syntactical and semantic similarity does not ensure that two programs produce identical runtime behavior. To address this issue, we use execution-based differential analysis. In this work we use CrossHair, a property-based testing tool that detects behavioral inconsistencies by generating counterexamples—inputs that cause two programs to produce different outputs. This allows us to evaluate functional equivalence beyond the static information computed with CodeBLEU. For each pair of candidate programs, EnsLLM runs CrossHair’s diffbehavior analysis, systematically exploring edge cases to identify discrepancies. The number of counterexamples serves as an inverse similarity measure:

- Zero counterexamples indicate that two programs are behaviorally identical, meaning they produce the same output for all tested inputs.
- One or more counterexamples indicate that the programs exhibit different behaviors, highlighting logic errors or functional differences.

To quantify behavioral similarity, we introduce a metric that evaluates the consistency of program outputs using differential analysis. The behavioral similarity metric is defined as follows:

$$\text{BSim}_n(P_i, P_j) = \left(1 - \frac{\text{MIN}(n, (\text{cex}(P_i, P_j)))}{n} \right) \quad (1)$$

where $\text{cex}(P_i, P_j)$ represents the number of counterexamples detected where the two programs produce different outputs. As the number of counterexamples can be arbitrarily large, we normalize their impact by capping it at n , a small natural number, preventing extreme values from disproportionately influencing the similarity score. This formulation ensures that the similarity decreases as behavioral inconsistencies increase, while the subtraction from 1 prioritizes programs with fewer functional differences.

By incorporating information about behavioral similarity, we aim to ensure that functionally inconsistent programs receive lower similarity scores, improving the reliability of the final program selection.

We use differential analysis to complement the computed CodeBLEU similarity. Since syntactic and semantic similarity (computed by CodeBLEU) alone do not guarantee identical input-output behavior, differential testing provides an additional layer of validation.

When two correct implementations, such as the iterative and recursive binary search functions (Listings 1 and 2), are compared, they consistently produce identical outputs for all valid inputs on a sorted list, resulting in no behavioral deviations. This confirms their correctness and reinforces their ranking in the selection process.

```

1 def binary_search_recursive_wrong(arr, target, left, right):
2     if left >= right:
3         return -1
4     mid = left + (right - left) // 2
5     if arr[mid] == target:
6         return mid
7     elif arr[mid] < target:
8         return binary_search_recursive_wrong(arr,
9                                               target, mid, right)
10    else:
11        return binary_search_recursive_wrong(arr,
12                                              target, left, mid)

```

Listing 4: Incorrect Recursive Binary Search Implementation

In contrast, incorrect implementations exhibit behavioral inconsistencies when compared against correct ones. For instance, the flawed recursive binary search (Listing 4) deviates from the standard binary search logic by improperly updating the search bounds, potentially leading to infinite recursion or missed target values. When the correct iterative (Listing 1) or recursive binary search (Listing 2) is compared against this incorrect version, differential analysis reveals two counterexamples with both Listings 1 and 2. Incorrect programs, when evaluated against correct implementations, consistently produce counterexamples, leading to lower rankings in the selection process. However, CrossHair (differential analysis) alone is not enough; e.g., it would not differentiate between Listings 1 and 2 vs. Listing 3, because they have the same input-output behaviour (but Listing 3 is not a desirable solution since it does not implement a binary search).

3.3 Integration of Syntactic, Semantic and Behavioral Similarity

We define our similarity metric by integrating syntactic, semantic, and behavioral equivalence using CodeBLEU and CrossHair-based differential analysis.

$$\text{similarity}(P_i, P_j) = \lambda \cdot \text{CodeBLEU}(P_i, P_j) + (1 - \lambda) \cdot \text{BSim}_n(P_i, P_j) \quad (2)$$

Here λ is a weighting factor that balances syntactic and semantic similarity (measured by CodeBLEU) and behavioral similarity (measured by BSim_n). CodeBLEU provides a normalized similarity score between 0 and 1, evaluating lexical, structural, and data-flow alignment between programs. BSim_n complements this by assessing functional correctness, ensuring that programs producing similar outputs are ranked higher.

CodeBLEU and CrossHair complement each other by capturing different aspects of correctness, ensuring a more reliable selection process. Although incorrect, Listing 4 appears more structurally similar to the correct implementations when evaluated using CodeBLEU. Since it follows the recursive binary search structure, it receives a relatively high syntax similarity score when compared with Listings 1 and 2. Additionally, its data flow similarity score remains moderate, as variable transformations in the recursive calls

Algorithm 1 Voting with Aggregated Pairwise Similarity

Require: List of programs $P = \{P_1, P_2, \dots, P_n\}$

Ensure: The program P^* with the highest aggregated similarity

```

1: for each  $P_i$  in  $P$  do
2:     for each  $P_j$  in  $P$  where  $i \neq j$  do
3:         similarity[i][j] ← Compute_Similarity( $P_i, P_j$ )
4:     end for
5: end for
6: for each  $P_i$  in  $P$  do
7:     agg_sim[i] ← 0
8:     for each  $P_j$  in  $P$  where  $i \neq j$  do
9:         agg_sim[i] ← agg_sim[i] + similarity[i][j]
10:    end for
11: end for
12: best_programs ← { $P_i$  | agg_sim[i] = max(agg_sim)}
13:  $P^* \leftarrow \text{tie\_breaking}(\text{best\_programs})$ 
14: return  $P^*$ 

```

resemble those in the correct implementation, even though they are flawed. This could lead to a higher aggregated CodeBLEU score in pairwise comparisons, potentially misleading the ranking if used in isolation.

However, CrossHair’s counterexample analysis helps mitigate this issue by directly verifying behavioral correctness. While Listing 3 is an entirely different algorithm and thus receives low syntax and data flow similarity scores in CodeBLEU, its simplicity aids CrossHair in validating expected versus actual outputs. Since linear search consistently finds the target, it does not produce any counterexample with Listing 1 and 2, but produces 2 counterexamples in differential analysis with Listing 4, which fails under certain cases due to incorrect index updates. This ensures that even among incorrect implementations, CrossHair effectively differentiates functional correctness. By integrating CodeBLEU’s structural and semantic similarity with CrossHair’s behavioral validation, our approach ensures that one of the correct implementations, Listing 2, is ranked highest. This synergy between CodeBLEU and CrossHair allows us to filter out misleadingly high-scoring but incorrect solutions while ensuring that the most robust implementation is selected.

3.4 Voting with Aggregated Similarity

Finally, for each candidate program, we compute an aggregated similarity score by summing its similarity scores with all the other programs in the set. This score quantifies the program’s alignment with the consensus among the generated candidates, ensuring that syntactically, semantically, and behaviorally consistent programs are prioritized.

$$\text{aggregated_similarity}(P_i) = \sum_{j \neq i} \text{similarity}(P_i, P_j) \quad (3)$$

Algorithm 1 shows our structured voting-based selection process to identify the most reliable program from a set of candidates. It first computes pairwise similarity between programs and stores the results in a similarity matrix (Lines 1 – 5). Then, it calculates each program’s aggregated similarity score by summing its similarity

values with all others (Lines 6 – 11). The program with the highest aggregated similarity will be selected as the best program. If multiple programs have the highest score, a tie-breaking function is applied to determine the final selection. In case of a tie in aggregated similarity, the number of CrossHair counterexamples is used as a tiebreaker, prioritizing the program with fewer counterexamples, and if still tied, a random selection ensures a deterministic choice while maintaining diversity in program selection.

Selecting the program with the highest aggregated similarity score ensures that the chosen solution is the most consistent, reliable, and error-resilient among all generated candidates. By prioritizing the program that aligns most closely with the consensus, we reduce the impact of outliers—programs that significantly deviate from common patterns due to syntax errors, logic flaws, or behavioral inconsistencies. Additionally, this approach enhances syntactical, semantical, and behavioral reliability, as the selected program not only shares key syntactic and semantic features with other high-quality candidates but also exhibits minimal behavioral differences, ensuring correct execution. Furthermore, programs that contain incorrect logic or unintended variations are naturally penalized with lower similarity scores, making them less likely to be chosen. As a result, our selection process effectively filters out unreliable outputs while reinforcing the selection of functionally correct and syntactically and semantically sound programs.

3.5 Limitations

While our approach significantly improves code generation accuracy, it has several limitations. First, if none of the candidate programs contain a correct solution, our method cannot generate a correct program on its own; it can only select from the available candidates. This makes its effectiveness highly dependent on the quality of the generated outputs from the LLMs. Also, if the assumption, that LLMs do not make identical mistakes, is wrong and multiple models generate similar incorrect outputs, our selection may fail to filter them out.

Second, if after the initial syntactical correctness filtering, only two programs remain, our pairwise similarity-based voting mechanism fails to differentiate between them, as they will always receive identical aggregated scores. This limitation arises because our approach relies on relative ranking rather than absolute correctness evaluation.

Third, in cases where multiple programs receive similar aggregated similarity scores, our tie-breaking mechanism prioritizes the program with fewer CrossHair counterexamples, but this does not always guarantee correctness. Since CrossHair only identifies behavioral differences based on its generated test cases, it may fail to capture deeper functional issues, leading to ambiguities in the final selection.

4 Evaluation

4.1 Experimental Setup

To evaluate EnsLLM we conduct extensive experiments on two well-established program generation benchmarks: HumanEval and LiveCodeBench. The goal is to assess the effectiveness of our ensemble approach compared to standalone Large Language Models (LLMs) in selecting functionally correct program.

4.1.1 Research Questions. Our evaluation is structured around three key research questions.

- **RQ1: How does EnsLLM perform compared to standalone LLMs?**

We measure functional correctness using pass@1 metrics and evaluate if EnsLLM improves the overall success rate over standalone models.

- **RQ2: How do CodeBLEU and CrossHair individually contribute to the accuracy of EnsLLM?**

An ablation study is conducted to assess the independent impact of CodeBLEU (syntactical and semantical similarity) and CrossHair (behavioral equivalence testing) in selecting correct outputs.

- **RQ3: How much accuracy can be gained using only free LLMs?**

We analyze whether EnsLLM can effectively leverage only open-source models (e.g., OpenChat, CodeLlama, DeepSeek-Coder) while maintaining high accuracy.

4.1.2 Benchmarks.

To rigorously evaluate EnsLLM, we utilize two widely recognized benchmarks: HumanEval and LiveCodeBench. These datasets provide diverse programming tasks that assess the ability of large language models (LLMs) to generate functionally correct and syntactically valid program.

- **HumanEval** [16] is a benchmark specifically designed for evaluating the functional correctness of LLM-generated code. It consists of 164 Python programming problems, each presented with a structured format that includes a natural language prompt, a function signature, and a set of hidden test cases used for verification. The problems span a variety of domains, including mathematical computations, string manipulations, data structures, and logical reasoning. Since the primary focus of HumanEval is on correctness rather than efficiency or complexity, the generated program is evaluated using the pass@k metric. This metric determines whether at least one of the top k generated solutions is functionally correct based on the provided test cases. Given its controlled setup and standardized evaluation framework, HumanEval has become a widely adopted benchmark in the automated code generation community.

- **LiveCodeBench** [51] is a comprehensive benchmark designed to assess the capabilities of LLMs across multiple programming related tasks. The dataset consists of four distinct categories: Code Generation, Self-Repair, Test Output Prediction, and Code Execution. For our evaluation, we focus exclusively on the Code Generation subset, which contains 511 programming problems covering a wide range of coding scenarios. These problems are more diverse than those in HumanEval, as they include real-world coding tasks, API usage, algorithmic challenges, and data structure manipulations. Additionally, LiveCodeBench introduces more complex problem formulations, requiring models to not only generate syntactically correct code but also ensure proper API usage and logical coherence. Like HumanEval, pass@k is used as the primary evaluation metric, providing a robust way to measure functional correctness. The inclusion of this

dataset ensures that EnsLLM is tested on a broader range of coding challenges that more accurately reflect real-world software development scenarios.

4.1.3 Baseline Models.

We compare EnsLLM against a diverse set of 14 LLMs, including both proprietary and open-source models:

- Proprietary LLMs: GPT-4o, GPT-4, GPT-3.5, CoPilot, Gemini
- Open-Source LLMs: OpenChat, CodeBERT, Llama 3.2, Qwen2, Codestral, Gemma 2, DeepSeekCoder, CodeLlama, DolphinCoder

Proprietary models (GPT-4o, GPT-4, GPT-3.5, CoPilot, and Gemini) are accessed via their official APIs, providing state-of-the-art code generation and reasoning capabilities. For open-source models (OpenChat, CodeBERT, Llama 3.2, Qwen2, Codestral, Gemma 2, DeepSeekCoder, CodeLlama, and DolphinCoder), we use the Ollama tool and the ollama-python library for execution. Open-source models offer flexibility, transparency, and offline usability, making them viable alternatives to commercial models. This diverse selection allows for a balanced comparison, evaluating both cutting-edge proprietary models and freely available alternatives in the context of code generation.

4.1.4 Implementation.

The implementation of EnsLLM follows a structured ensembling process that aggregates outputs from multiple Large Language Models (LLMs) to enhance the reliability and correctness of generated program. The process begins with the collection of candidate solutions from 14 different LLMs, each independently generating responses to a given programming problem. This diversity in generation provides a broad spectrum of possible correct implementations, improving the chances of selecting a high-quality solution. However, before proceeding to selection, all generated programs undergo syntax validation using PyLint, ensuring that syntactically invalid solutions are filtered out, thereby reducing erroneous candidates early in the pipeline.

To determine the most functionally correct program, EnsLLM employs a similarity-based selection approach, combining CodeBLEU for syntactic and semantic similarity analysis with CrossHair for behavioral correctness evaluation. CodeBLEU assesses the lexical, syntactic, and semantic alignment between different candidate solutions by analyzing their Abstract Syntax Tree (AST) and data flow. This ensures that solutions syntactically and semantically similar to high-quality implementations are ranked higher. Meanwhile, CrossHair complements this by performing counterexample-based behavioral analysis, identifying discrepancies in function outputs when executed with varying inputs. Solutions that remain consistent under different test cases receive a higher ranking. The final selection is based on an aggregated similarity score that integrates these two evaluation mechanisms, ensuring that the chosen solution aligns with both syntactical and semantical correctness and functional reliability. For this experiment, EnsLLM is implemented with the value of $\lambda = .5$ and $n = 10$.

To verify the correctness of the selected solution, EnsLLM executes it against unit test cases provided in the benchmark datasets. Each programming problem in HumanEval and LiveCodeBench is associated with a set of predefined test cases that assess whether

Table 1: Performance of EnsLLM compared to standalone LLMs on HumanEval and LiveCodeBench (pass@1). (F) denotes the free LLMs, and the accuracy provided in the braces is achievable accuracy.

LLM	HumanEval (%)	LiveCodeBench (%)
GPT-4o	83.5	43.4
GPT-4	82.3	42.2
GPT-3.5	76.8	39.1
CoPilot	74.4	41.6
OpenChat (F)	71.3	37.3
CodeBert (F)	68.9	37.2
Llama 3.2 (F)	68.9	36.7
Gemini	66.5	36.1
Qwen2 (F)	62.1	36.4
Codestral (F)	59.2	37.2
Gemma 2 (F)	52.1	31.3
DeepSeekCoder (F)	50.6	25.6
CodeLlama (F)	42.7	23.4
DolphinCoder (F)	34.8	22.2
EnsLLM (All)	90.2 (90.9)	50.2 (53.8)
EnsLLM (Top 5)	87.2 (90.9)	48.3 (53.8)
EnsLLM (All Free)	80.5 (83.2)	41.6 (44.1)

the generated program produces the expected outputs. In this evaluation, we have used pass@1 as the primary metric, meaning that for each programming problem, we generate only one program solution from an LLM. This metric evaluates the probability that the single generated solution is functionally correct by executing it against the predefined unit test cases in the benchmark datasets. Using pass@1 provides a strict and realistic measure of performance, as it reflects the model's ability to generate a correct solution on the first attempt without relying on multiple outputs. This approach aligns well with real-world coding scenarios, where developers typically seek an immediately functional solution rather than generating multiple alternatives for a single problem. The source code is available at <https://github.com/TSUMahmud/EnsLLM>.

All experiments are conducted on a high-performance computing system equipped with an NVIDIA GeForce RTX 4060 (16GB VRAM), 32GB RAM, and an Intel Core i7 (12th Gen) processor running Windows 11. The use of GPU acceleration ensures the efficient execution of multiple LLMs, significantly reducing computation time for large-scale benchmarking tasks. This setup allows for scalable and reproducible experimentation, ensuring that EnsLLM's performance is rigorously evaluated under realistic conditions.

4.2 Experimental Results

4.2.1 RQ1: How does EnsLLM perform compared to standalone LLMs?

To evaluate the effectiveness of EnsLLM, we compare its performance against standalone LLMs on the HumanEval and LiveCodeBench datasets using the pass@1 metric. The goal is to determine whether EnsLLM improves accuracy over individual models by leveraging an ensemble-based selection strategy.

Table 1 presents the results of individual LLMs as well as EnsLLM's performance on HumanEval and LiveCodeBench in 3 modes. The EnsLLM (All) mentioned the results of EnsLLM when using all

the LLMs considered in this study. Achievable accuracy, mentioned in the braces with the accuracy of EnsLLM, refers to the maximum possible accuracy that the ensemble-based selection approach can attain, assuming an optimal selection strategy. It is defined as the proportion of problems for which at least one of the individual LLMs in the ensemble produces a correct solution. Since EnsLLM does not generate new program but rather selects the best candidate from multiple LLM outputs, its upper bound performance is inherently constrained by the presence of correct solutions among the generated candidates. Mathematically, if an ensemble of N LLMs is used, and for a given benchmark dataset, there are M total problems and C is the number of problems where at least one LLM generated a correct solution the achievable accuracy is calculated by:

$$\frac{C}{M} \times 100$$

This metric provides a theoretical limit on how well EnsLLM can perform. In our evaluation, this upper bound is 90.9% for HumanEval and 53.8% for LiveCodeBench, meaning that even with a perfect selection mechanism, the system cannot exceed these accuracy values since no correct solution exists for the remaining problems in the set of LLM-generated outputs.

The best-performing standalone models on HumanEval are GPT-4o (83.5%), followed closely by GPT-4 (82.3%) and GPT-3.5 (76.8%). Similarly, for LiveCodeBench, the highest accuracy among standalone models is GPT-4o (43.4%), with GPT-4 (42.2%) and CoPilot (41.6%) ranking next. EnsLLM, achieved 90.2% accuracy on HumanEval, surpassing all standalone models, including GPT-4o, which achieved the highest accuracy among individual LLMs at 83.5%. This result demonstrates that by leveraging multiple models and selecting the most reliable solution, EnsLLM can exceed the performance of any single LLM. Additionally, EnsLLM achieved 50.2% accuracy on LiveCodeBench, significantly outperforming the best standalone model, GPT-4o, which reached 43.4%. Given the challenging nature of LiveCodeBench problems, this improvement highlights the effectiveness of ensemble-based selection in handling diverse and complex coding tasks.

By integrating multiple LLMs and applying a structured ensembling strategy, EnsLLM provides substantial improvements over individual models, particularly in LiveCodeBench, where it achieves nearly 7 percentage points higher accuracy than the best-performing standalone model. As shown in Table 1, EnsLLM consistently outperforms all standalone LLMs, demonstrating that an ensemble-based selection approach can significantly enhance functional correctness in code generation. The results further emphasize that while standalone LLMs are constrained by their individual capabilities, EnsLLM effectively combines their strengths, leading to higher accuracy and more reliable code generation across diverse problem domains.

To further analyze the impact of model selection, we evaluated an alternative configuration, EnsLLM (Top 5), which selects the best-performing subset of five models rather than using all available LLMs. This setup achieves an accuracy of 87.2% on HumanEval and 48.3% on LiveCodeBench, compared to the achievable accuracy of 90.9% and 53.8%, respectively. The results show that even with a limited subset of models, EnsLLM (Top 5) is able to approach the upper bound of achievable accuracy; on the other hand, the

Table 2: Ablation study showing the contribution of CodeBLEU and CrossHair to accuracy.

λ	HumanEval (%)	LiveCodeBench (%)
1	85.9	49.5
.75	87.2	49.5
.5	90.2	50.2
.25	89.6	48.4
0	89.6	47.9

results indicate that the less performant models – not present in the EnsLLM (Top 5) but present in EnsLLM (All) – do have a significant contribution in the performance of the ensemble.

4.2.2 RQ2: How do CodeBLEU and CrossHair individually contribute to the accuracy of EnsLLM?

Equation 2 defines the unified similarity metric, which integrates CodeBLEU and behavioral similarity (BSim_n) using a weighting factor λ to balance syntactic/semantic similarity and behavioral equivalence. CodeBLEU captures lexical, syntactical, and semantic alignment, while BSim_n evaluates functional correctness by measuring behavioral differences between candidate programs.

To determine the optimal balance between CodeBLEU and BSim_n , we conducted an ablation study by varying λ and evaluating accuracy on HumanEval and LiveCodeBench (Table 2). The results show that $\lambda = 0.5$ achieves the highest accuracy on both datasets, with 90.2% on HumanEval and 50.2% on LiveCodeBench. This suggests that giving equal weight to syntactic/semantic similarity and behavioral similarity yields the most robust and reliable selection of generated programs.

When $\lambda = 1$ (fully relying on CodeBLEU), accuracy drops to 85.9% on HumanEval and 49.5% on LiveCodeBench, indicating that ignoring behavioral correctness leads to functional errors despite structural similarity. Conversely, when $\lambda = 0$ (fully relying on BSim_n), accuracy reaches 89.6% on HumanEval but declines to 47.9% on LiveCodeBench, showing that behavioral checks alone are insufficient without syntactic/semantic alignment. These results confirm that both structural similarity and behavioral correctness are essential, and an equal balance between the two provides the best overall performance.

To gain deeper insights into the impact of CodeBLEU, we explored various weighting strategies for its four similarity components: lexical similarity, syntactic similarity, data-flow similarity, and token importance weighting. We systematically tested all possible combinations of these weights to determine their influence on accuracy. Table 3 presents only the configurations that yielded better results compared to the baseline, where all four components were equally weighted at 0.25 for both datasets. When all four CodeBLEU matrices were equally weighted at 0.25, the accuracy was 85.4% on HumanEval and 46.1% on LiveCodeBench. This indicates that while distributing the weights evenly provides reasonable accuracy, it may not be optimal for all problem types.

By adjusting the weights to favor syntactic and data-flow similarity, where $\text{syntax_weight} = 0.5$ and $\text{dataflow_weight} = 0.5$, accuracy improved to 89.0% on HumanEval and 49.5% on LiveCodeBench. This suggests that prioritizing syntactic alignment and semantic equivalence results in better selection performance, as these aspects

Table 3: Impact of different CodeBLEU weighting configurations on accuracy.

Settings	HumanEval (%)	LiveCodeBench (%)
syntax_weight = 0.25, dataflow_weight = .25, n_gram_weight = 0.25, token_weight = 0.25	85.4	46.1
syntax_weight = 0.5, dataflow_weight = .25, n_gram_weight = 0.25	86.0	46.4
syntax_weight = 0.25, dataflow_weight = .5, n_gram_weight = 0.25	86.6	46.4
syntax_weight = 0.25, dataflow_weight = 0.75	89.0	49.5
syntax_weight = 0.5, dataflow_weight = 0.5	89.0	49.5

more accurately capture correct implementations. Similarly, using `syntax_weight = 0.25` and `dataflow_weight = 0.75` produced similar results, reinforcing that placing greater emphasis on semantic correctness improves overall accuracy.

These results demonstrate that lexical similarity and token importance weighting contribute less to correctness verification, whereas syntax and data-flow analysis are critical for selecting high-quality code. Reducing the influence of lexical similarity prevents the model from favoring textually similar but functionally incorrect solutions, improving the overall robustness of EnsLLM.

The answer to this research question confirms that CrossHair is highly effective for ensuring behavioral correctness, while CodeBLEU is crucial for ranking semantically similar and syntactically well-formed programs. However, using both together yields the best performance, as each metric complements the other. Additionally, optimizing CodeBLEU’s weight distribution—by prioritizing syntax and data-flow similarity over lexical similarity—further improves accuracy.

To determine the optimal value of n , we conducted experiments by varying n from 5 to 10 and analyzing its impact on accuracy. The results showed that setting n between 6 and 10 yielded the highest accuracy, while choosing n below 6 led to a noticeable decline in performance. Specifically, for $n < 6$, the accuracy dropped to 89.2%, indicating that a lower n fails to capture sufficient behavioral differences, reducing the effectiveness of the selection process.

These findings reinforce that a combined approach leveraging both syntactic and behavioral correctness is essential for achieving the highest accuracy in EnsLLM. By fine-tuning CodeBLEU weights and integrating CrossHair for functional validation, EnsLLM effectively enhances code generation reliability across diverse problem domains.

4.2.3 RQ3: How much accuracy can be gained using only free LLMs?

To evaluate the effectiveness of EnsLLM when restricted to using only free LLMs, we conducted experiments using an ensemble of open-source models rather than proprietary ones like GPT-4o and CoPilot. The goal was to determine whether EnsLLM can still achieve competitive performance without relying on high-resource commercial models.

The results, presented in Table 1, show that when using only free models, the highest-performing standalone model is OpenChat, which achieves an accuracy of 71.3% on HumanEval and 37.3% on LiveCodeBench. The achievable accuracy, which represents the upper bound where the best free model is selected for each problem, is 83.2% for HumanEval and 44.1% for LiveCodeBench. This indicates that while free models individually underperform compared to proprietary models, an optimal selection strategy can still yield reasonably high accuracy.

EnsLLM achieves an accuracy of 80.5% on HumanEval and 41.6% on LiveCodeBench using only free models. This demonstrates that even without access to high-end proprietary models, EnsLLM can still significantly improve accuracy by leveraging an ensemble of diverse open-source LLMs. Compared to the best individual free model, EnsLLM gains nearly 9 percentage points on HumanEval and 4.3 percentage points on LiveCodeBench, highlighting the effectiveness of our ensembling approach in improving functional correctness.

The results confirm that EnsLLM can still achieve strong performance using only free models, significantly outperforming individual open-source LLMs. While it does not reach the accuracy of proprietary model ensembles, it remains a viable alternative when commercial models are unavailable. The ensemble selection process is key to improving accuracy, as it compensates for the weaker performance of individual free models by strategically selecting the most functionally correct solutions. This demonstrates that ensemble-based code generation approaches can enhance reliability even in resource-constrained settings, making them practical for scenarios where access to proprietary models is limited.

However, EnsLLM’s accuracy in this setting remains lower than the full ensemble (which includes proprietary models) due to the limited capabilities of free models, particularly in handling complex coding tasks in LiveCodeBench. The gap between EnsLLM’s performance (80.5%) and the achievable accuracy (83.2%) in HumanEval suggests that further optimizations in model selection and weighting strategies could narrow this difference.

5 Discussions

5.1 Qualitative Analysis

To better understand the behavior of EnsLLM we conducted a qualitative analysis of the failed cases on the HumanEval benchmark. Out of 164 problems, we found that in 131 cases, at least two individual models independently generated correct solutions. In all of these instances, EnsLLM successfully selected the correct program. This strongly supports our hypothesis that correct outputs reinforce one another in our similarity-based ensemble, making them more likely to be chosen.

For 18 problems (out of the rest 33 problems), only a single model produced the correct solution for each of those problems. Even in these more challenging scenarios, EnsLLM correctly identified the correct program in 17 cases, as no incorrect programs made similar mistakes that would have reinforced one another. The one failure occurred when three models produced incorrect programs with similar mistakes, leading EnsLLM to select the wrong output. This illustrates a key limitation of the approach: when multiple models converge on similar errors, the ensemble may amplify rather than

Table 4: Performance of EnsLLM compared to standalone LLMs on HumanEval-Java(pass@1). The accuracy provided in the braces is achievable accuracy.

LLM	Accuracy (%)
GPT-4o	90
GPT-4	84
GPT-3.5	78
CoPilot	86
OpenChat (F)	70
Llama 3.2 (F)	72
Gemini	86
Qwen2 (F)	72
Codestral (F)	68
Gemma 2 (F)	62
DeepSeekCoder (F)	60
CodeLlama (F)	64
DolphinCoder (F)	48
EnsLLM (All)	90 (90)
EnsLLM (Top 5)	90 (90)
EnsLLM (All Free)	78 (78)

avoid them. However, such failures were rare and further support the assumption that diverse model behaviors reduce correlated mistakes. For the remaining 15 cases, no correct solutions were available for EnsLLM to select.

5.2 Cost and Scalability

Collecting outputs sequentially from each of the LLMs takes 94.22 seconds per problem on average, with an associated cost of approximately 0.005 USD per proprietary model. While parallel execution can significantly reduce latency, running all 14 models, especially at scale, poses practical challenges due to resource demands and infrastructure constraints. The ensemble evaluation stage adds further overhead: computing all 91 pairwise similarity scores using CodeBLEU takes 86.21 seconds per problem, and running behavioral checks with CrossHair requires 212.38 seconds. While the overhead is non-negligible, EnsLLM can be viewed as trading this cost for improved reliability in the code generated by LLMs. In future work, we plan to explore strategies for more efficient ensemble execution, including selective model invocation, partial ensembling, or model pruning. We also aim to investigate system-level solutions for running multiple LLMs concurrently with minimal resource overhead, to make the approach more practical for real-world deployment.

5.3 Case Study on HumanEval-Java

To demonstrate the generalizability of EnsLLM, we conducted a case study on the HumanEval-Java [1] benchmark, which closely follows the structure and semantics of HumanEval but is written in Java. For syntactic and semantic similarity, we reused CodeBLEU, which supports multi-language analysis without requiring modification. However, since CrossHair is Python-specific, an alternative approach was needed to approximate behavioral similarity for Java programs.

We employed Kex [2], a symbolic execution-based test generation framework for Java, as a substitute for CrossHair in the Java

setting. Kex performs path-sensitive analysis by symbolically executing Java bytecode to explore multiple execution paths and automatically generates their corresponding input values. For each candidate program P_i , we used Kex to generate a suite of test cases, with the aim to cover the program’s behaviors without requiring manually written specifications.

To compute behavioral similarity between two programs P_i and P_j , we leveraged these generated test cases in a cross-execution strategy. Specifically, for every program pair (P_i, P_j) , we executed the test suite synthesized for P_i on the implementation of P_j . We then counted the number of test cases that failed when run against P_j . This count was denoted as $cex(P_i, P_j)$ and treated as a proxy for behavioral divergence from P_i ’s semantics.

To convert this into a normalized similarity measure, we applied the same formulation used in Equation (1), capping the counterexamples at $n = 6$ to avoid an extreme penalty due to overly sensitive or noisy test cases. Due to the significant effort required to integrate this new workflow, we report here on a preliminary evaluation on the first 50 problems from the HumanEval-Java dataset. We exclude CodeBERT from this experiment as its performance is hindered by the prompt structure used in HumanEval-Java. Unlike benchmarks such as HumanEval or LiveCodeBench, HumanEval-Java lacks well-researched prompt engineering, leading CodeBERT, a masked language model, to return the input unchanged, failing to generate meaningful method bodies.

As shown in Table 4, EnsLLM achieved an accuracy of 90%, which matches the achievable accuracy. Although the accuracy of the best-performing standalone model, GPT-4o, is also 90%, EnsLLM selected the correct solution in every case where a correct solution was available. With the top 5 or all free models, EnsLLM also matches the achievable accuracy. With all free models, EnsLLM’s accuracy (78%) is higher than the best performing models Llama 3.2 and Qwen2 (72%). These findings suggest that the core principles of our ensemble method generalize well across programming languages, provided that compatible analysis tools are available or can be adapted.

5.4 Threats to Validity

The main threat to internal validity is CrossHair’s effectiveness depends on the complexity of generated programs; if a program involves external dependencies or non-deterministic behavior, CrossHair may fail to detect functional discrepancies. To mitigate these risks, we integrate both CodeBLEU and CrossHair, ensuring that correct implementations reinforce each other while incorrect ones are penalized.

For external validity, our evaluation is limited to HumanEval and LiveCodeBench, which may not fully represent complex, real-world programming challenges. However, these two benchmarks are widely recognized in code generation research and have been used in several similar studies to evaluate the functional correctness of LLM-generated programs. Their extensive use ensures a standardized comparison with existing approaches, mitigating the threat to generalizability. A potential threat to external validity lies in the selection of the 14 LLMs used in our ensemble. While we included a mix of open-source and proprietary models with varied sizes and tuning, the selection was not based on a formal

strategy. This may limit the generalizability of our results, particularly regarding robustness to correlated errors. Future work will explore principled selection methods based on model diversity and behavioral variance to strengthen ensemble design. Another external validity concern is the potential presence of the solution of the datasets in the pretraining sets of some LLMs. While we cannot verify the training data of all models, any such overlap could inflate performance. Although our focus is on the ensemble mechanism rather than generation quality, this remains a limitation.

A potential threat to validity in our study is bias in language model selection, as the set of 14 LLMs we evaluate may not fully represent the entire spectrum of code generation models. While we include a diverse mix of proprietary and open-source models, newer or more specialized models may perform differently, potentially affecting our findings. To mitigate this, we select widely used models, ensuring our comparison aligns with existing research and industry practices. However, future work will expand this evaluation to a broader set of models to further validate generalizability.

6 Related Work

Large Language Models (LLMs) have significantly advanced automated code generation, enabling models to generate functionally correct code from natural language prompts. Early research introduced OpenAI Codex, a GPT-based model fine-tuned for code generation, which demonstrated strong capabilities in solving programming problems [16]. Codex was evaluated on the HumanEval benchmark, achieving a 28.8% success rate on pass@1 and 70.2% on pass@100, highlighting the benefits of generating multiple outputs and selecting the best one. This approach led to the development of tools like GitHub Copilot, which integrates LLM-based code generation into modern IDEs [42].

Despite these advances, LLMs still exhibit major limitations in correctness, reliability, and efficiency. Studies have shown that even state-of-the-art models like GPT-4 achieve only 82% correctness on HumanEval, while open-source models such as CodeLlama perform significantly worse, with accuracy ranging between 42-50% [13, 53]. LLMs often generate program that appears syntactically correct but contains logical errors, missing imports, or faulty API usage. Another challenge is security—a study on GitHub Copilot found that 40% of AI-generated program contained vulnerabilities, raising concerns about blindly adopting LLM outputs in production codebases [42].

To mitigate these issues, research has explored strategies such as reinforcement learning from human feedback (RLHF), filtering generated program based on static analysis tools, and incorporating behavioral validation techniques to assess correctness [22, 27]. In competitive programming, DeepMind’s AlphaCode demonstrated that LLMs can tackle algorithmically complex problems by generating thousands of solutions and filtering them based on test execution results. AlphaCode was able to achieve human-level performance in Codeforces competitions, ranking among the top 54% of human participants [36]. This suggests that while LLMs still require validation and post-processing, their ability to generate correct program improves significantly when paired with structured filtering and selection methods. Our work differs by introducing an ensemble-based approach that selects the most reliable program

from multiple LLM-generated outputs using CodeBLEU for syntactic and semantic similarity and CrossHair for behavioral validation.

Beyond code generation, LLMs have demonstrated impressive performance across various natural language processing and coding-related tasks, including code completion [19, 28, 30, 36, 50], code synthesis [37], and program repair [48]. Codex [9], the model behind GitHub Copilot [7], has shown promise in automatically translating comments into functional code, significantly improving developer productivity, though minor issues persist. Other LLMs, such as BERT [20, 46], GPT [6, 8, 49], and various domain-specific models [35, 41], have also proven their ability to generate syntactically correct and contextually relevant program.

LLMs have also been explored in broader software engineering tasks, including automated bug detection, test case generation, code summarization, and security vulnerability analysis [27, 31]. For instance, they can assist in writing unit tests by generating test cases from function signatures, improving test coverage while reducing developer effort [26]. Additionally, LLMs are being integrated into code refactoring tools, where they suggest optimizations and enhance code readability [23]. Beyond these applications, LLMs are transforming software engineering workflows, influencing how software is developed [29, 38], enhancing developer productivity [18, 52], and assisting in security analysis [43]. However, challenges persist in ensuring correctness, maintainability, and security, requiring further refinement and validation before LLM-generated code can be reliably integrated into real-world software systems.

Test-based selection has been explored to improve code generation. CodeT [15] uses LLM-generated tests to rank candidate programs from the same model based on execution results and agreement. Fakhoury et al. [21] similarly apply unit test feedback in an interactive loop to guide code refinement. While conceptually aligned with our goal of functional selection, these methods rely on a single model, limiting candidate diversity. On the other hand, EnsLLM leverages multiple LLMs, reducing correlated errors and enabling more robust selection.

7 Conclusion

In this paper, we introduced EnsLLM, a structured ensemble-based approach to improve the reliability and accuracy of LLM-generated code. By leveraging pairwise similarity scoring through CodeBLEU for syntactic and semantic alignment and CrossHair-based differential analysis for behavioral consistency, our method systematically selects the most functionally correct and syntactically sound program from multiple LLM-generated candidates. Our evaluation on HumanEval and LiveCodeBench demonstrates that EnsLLM consistently outperforms standalone LLMs, which highlights the potential of structured ensembling in LLM-based code generation.

In future work, we plan to extend EnsLLM with more code quality metrics, related to security, performance and maintainability, paving the way for more reliable, adaptable, and robust AI-assisted programming workflows.

Acknowledgments

This work was partially supported by The University of Queensland NSRSG Grant NS-2201 and Oracle Labs.

References

- [1] [n. d.]. humaneval-java. <https://huggingface.co/datasets/Dataset678/humaneval-java>.
- [2] [n. d.]. Kex. <https://github.com/vorpai-research/kex>.
- [3] 2020. CrossHair: Symbolic Execution for Python. <https://github.com/pschaneley/CrossHair>.
- [4] 2022. CodeLlama. <https://codellama.dev/about>.
- [5] 2022. DeepseekCoder. <https://deepseekcoder.github.io/t>.
- [6] 2023. ChatGPT. <https://chat.openai.com/>.
- [7] 2023. GitHub Copilot. <https://github.com/features/copilot>.
- [8] 2023. GPT-4. <https://openai.com/gpt-4>.
- [9] 2023. OpenAI Codex. <https://openai.com/blog/openai-codex>.
- [10] Charu C Aggarwal. 2016. Outlier ensembles. In *Outlier Analysis*. Springer, 185–218.
- [11] Leo Breiman. 1996. Bagging predictors. *Machine learning* 24, 2 (1996), 123–140.
- [12] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [13] Sébastien Bubeck, Varun Chandrasekaran, Ron Eldan, Johannes Gehrmke, Eric Horvitz, Ece Kamar, Peter Lee, Or Levy, Yin Tat Li, Scott Lundberg, et al. 2023. Sparks of Artificial General Intelligence: Early Experiments with GPT-4. *arXiv preprint arXiv:2303.12712* (2023).
- [14] Rich Caruana, Alexandru Niculescu-Mizil, Geoff Crew, and Alex Ksikes. 2004. Ensemble selection from libraries of models. In *Proceedings of the twenty-first international conference on Machine learning*. 18.
- [15] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. 2022. CodeT5: Code generation with generated tests. *arXiv preprint arXiv:2207.10397* (2022).
- [16] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).
- [17] Tianqi Chen and Carlos Guestrin. 2016. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*. 785–794.
- [18] Arghavan Moradi Dakhel, Vahid Majdinasab, Amin Nikanjam, Foutse Khomh, Michel C Desmarais, and Zhen Ming Jack Jiang. 2023. Github copilot ai pair programmer: Asset or liability? *Journal of Systems and Software* 203 (2023), 111734.
- [19] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. 2023. Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models. In *Proceedings of the 32nd ACM SIGSOFT international symposium on software testing and analysis*. 423–435.
- [20] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. 4171–4186.
- [21] Sarah Fakhoury, Aaditya Naik, Georgios Sakkas, Saikat Chakraborty, and Shuvendu K Lahiri. 2024. Llm-based test-driven interactive code generation: User study and empirical evaluation. *IEEE Transactions on Software Engineering* (2024).
- [22] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M Zhang. 2023. Large language models for software engineering: Survey and open problems. In *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*. IEEE, 31–53.
- [23] Lingling Fan, Ting Su, Sen Chen, Guozhu Meng, Yang Liu, Lihua Xu, Geguang Pu, and Zhendong Su. 2018. Large-scale analysis of framework-specific exceptions in Android apps. In *2018 IEEE/ACM 40th International Conference on Software Engineering (ICSE)*. IEEE, 408–419.
- [24] Yoav Freund and Robert E Schapire. 1997. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of computer and system sciences* 55, 1 (1997), 119–139.
- [25] Yoav Freund and Robert E Schapire. 1997. A decision-theoretic generalization of on-line learning and an application to boosting. *J. Comput. System Sci.* 55, 1 (1997), 119–139.
- [26] Stanka Hadzhikoleva, Todor Rachovski, Ivan Ivanov, Emil Hadzhikolev, and Georgi Dimitrov. 2024. Automated test creation using large language models: A practical application. *Applied Sciences* 14, 19 (2024), 9125.
- [27] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 1–79.
- [28] Qing Huang, Zhiqiang Yuan, Zhenchang Xing, Xiwei Xu, Liming Zhu, and Qinghua Lu. 2022. Prompt-tuned code language model as a neural knowledge base for type inference in statically-typed partial code. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–13.
- [29] Saki Imai. 2022. Is github copilot a substitute for human pair-programming? an empirical study. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*. 319–321.
- [30] Naman Jain, Skanda Vaidyanath, Arun Iyer, Nagarajan Natarajan, Suresh Parthasarathy, Sriram Rajamani, and Rahul Sharma. 2022. Jigsaw: Large language models meet program synthesis. In *Proceedings of the 44th International Conference on Software Engineering*. 1219–1231.
- [31] Haolin Jin, Linghan Huang, Haipeng Cai, Jun Yan, Bo Li, and Huaming Chen. 2024. From llms to llm-based agents for software engineering: A survey of current, challenges and future. *arXiv preprint arXiv:2408.02479* (2024).
- [32] Josef Kittler, Mohamad Hatef, Robert P W Duin, and Jiri Matas. 1998. On combining classifiers. In *IEEE Transactions on Pattern Analysis and Machine Intelligence*, Vol. 20. IEEE, 226–239.
- [33] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. 2012. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems* 25 (2012).
- [34] Shuvendu K Lahiri, Kapil Vaswani, and C AR Hoare. 2010. Differential static analysis: opportunities, applications, and challenges. In *Proceedings of the FSE/SDP workshop on Future of software engineering research*. 201–204.
- [35] Thanh Le-Cong, Duc-Minh Luong, Xuan Bach D Le, David Lo, Nhat-Hoa Tran, Bui Quang-Huy, and Quyet-Thang Huynh. 2023. Invalidator: Automated patch correctness assessment via semantic and syntactic reasoning. *IEEE Transactions on Software Engineering* (2023).
- [36] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-level code generation with alphacode. *Science* 378, 6624 (2022), 1092–1097.
- [37] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Advances in Neural Information Processing Systems* 36 (2023), 21558–21572.
- [38] Zhe Liu, Chunyang Chen, Junjie Wang, Xing Che, Yuekai Huang, Jun Hu, and Qing Wang. 2023. Fill in the blank: Context-aware automated text input generation for mobile gui testing. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1355–1367.
- [39] Ibomoye Domor Mienye and Yanxia Sun. 2022. A survey of ensemble learning: Concepts, algorithms, applications, and prospects. *Ieee Access* 10 (2022), 99129–99149.
- [40] Yannic Noller, Corina S. Păsăreanu, Marcel Böhme, Youcheng Sun, Hoang Lam Nguyen, and Lars Grunske. 2020. HyDiff: Hybrid Differential Software Analysis. In *2020 IEEE/ACM 42nd International Conference on Software Engineering (ICSE)*. 1273–1285.
- [41] Rishov Paul, Md Mohib Hossain, Masum Hasan, and Anindya Iqbal. 2023. Automated program repair based on code review: How do pre-trained transformer models perform? *CoRR* (2023).
- [42] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2025. Asleep at the keyboard? assessing the security of github copilot's code contributions. *Commun. ACM* 68, 2 (2025), 96–105.
- [43] Hammond Pearce, Benjamin Tan, Baleegh Ahmad, Ramesh Karri, and Brendan Dolan-Gavitt. 2023. Examining zero-shot vulnerability repair with large language models. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2339–2356.
- [44] Shiqi Ren, Zhiyang Tu, Deng Cai, Jun Zhang, and Yanzhi Chen. 2020. CodeBLEU: a method for automatic evaluation of code synthesis. In *Proceedings of the 28th International Conference on Computational Linguistics (COLING)*. 3981–3992.
- [45] Robert E Schapire and Yoram Singer. 1998. Improved boosting algorithms using confidence-rated predictions. In *Proceedings of the eleventh annual conference on Computational learning theory*. 80–91.
- [46] Haoye Tian, Kui Liu, Yinghua Li, Abdoul Kader Kaboré, Anil Koyuncu, Andrew Habib, Li Li, Junhao Wen, Jacques Klein, and Tegawendé F Bissyandé. 2023. The Best of Both Worlds: Combining Learned Embeddings with Engineered Features for Accurate Prediction of Correct Patches. *ACM Transactions on Software Engineering and Methodology* 32, 4 (2023), 1–34.
- [47] David H Wolpert. 1992. Stacked generalization. In *Neural networks*, Vol. 5. Elsevier, 241–259.
- [48] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated program repair in the era of large pre-trained language models. In *Proceedings of the 45th International Conference on Software Engineering (ICSE 2023)*. Association for Computing Machinery.
- [49] Chunqiu Steven Xia and Lingming Zhang. 2023. Keep the Conversation Going: Fixing 162 out of 337 bugs for \$0.42 each using ChatGPT. *arXiv preprint arXiv:2304.00385* (2023).
- [50] Frank F Xu, Uri Alon, Graham Neubig, and Vincent Josua Hellendoorn. 2022. A systematic evaluation of large language models of code. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*. 1–10.
- [51] Frank F Xu, Joon Sung Shin, Gabriel Poesia, Yao Luan, Pengcheng Yin, Xi Victoria Lin, Yi-Ting Chiu, Scott Lundberg, Hannaneh Hajishirzi, Mike Guo, et al. 2023. LiveCodeBench: Benchmarking Large Language Models for Code in the Wild. *arXiv preprint arXiv:2308.11233* (2023).

- [52] Albert Ziegler, Eirini Kalliamvakou, X Alice Li, Andrew Rice, Devon Rifkin, Shawn Simister, Ganesh Sittampalam, and Edward Aftandilian. 2022. Productivity assessment of neural code completion. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*. 21–29.
- [53] Mateusz Ziemniak, Johannes Meyer, Victor Bajona, Asya Glaese, Guillaume Belrose, Sebastian Borgeaud, Mayur Daswani, Luca Demontis, Marylou Gabri  , Felix Hill, et al. 2023. CodeLlama: Open Foundation Models for Code. *arXiv preprint arXiv:2308.12950* (2023).